Fine-Tuning ANN (from scratch) pada CIFAR-10 Dataset

Mata Kuliah: Pembelajaran Mendalam (IF25-40401) **Tugas:** Studio Proyek 1 — Kelompok

Nama Anggota 1: *(Ahmat Prayoga Sembiring)* **Catatan penting:** Notebook ini WAJIB dijalankan penuh di Google Colab (GPU T4) agar seluruh angka (loss, akurasi, waktu training, confusion matrix) merupakan hasil eksekusi nyata, bukan hasil karangan. Semua sel di bawah sudah siap dijalankan berurutan dari atas ke bawah (Runtime > Run all), lalu isi bagian *Analisis* dan *Kesimpulan* berdasarkan output yang benar-benar muncul.

1. Import Library

Mengimpor seluruh library yang dibutuhkan: PyTorch untuk membangun & melatih model, torchvision untuk dataset CIFAR-10, serta numpy/pandas/matplotlib/scikit-learn untuk analisis dan visualisasi.

```
import os
import time
import copy
import random

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, random_split

import torchvision
import torchvision.transforms as transforms

from sklearn.metrics import confusion_matrix, classification_report

print("Versi PyTorch :", torch.__version__)
print("Versi Torchvision:", torchvision.__version__)
```

```
Versi PyTorch : 2.11.0+cu128
Versi Torchvision: 0.26.0+cu128
```

2. Reproducibility / Random Seed

Agar hasil eksperimen dapat direproduksi (split data, augmentasi, dan inisialisasi bobot model sama setiap kali dijalankan), kita mengunci seed untuk semua sumber randomness (Python, NumPy, PyTorch).

```
SEED = 42

def set_seed(seed=SEED):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)
    torch.backends.cudnn.deterministic = True
    torch.backends.cudnn.benchmark = False

set_seed(SEED)
print(f"Seed dikunci ke {SEED}")
```

```
Seed dikunci ke 42
```

3. Cek Device dan GPU

Memastikan notebook menggunakan GPU (Colab T4) jika tersedia. Training ANN pada CIFAR-10 jauh lebih cepat dengan GPU dibanding CPU.

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Device yang digunakan:", device)
if torch.cuda.is_available():
    print("Nama GPU:", torch.cuda.get_device_name(0))
```

```
Device yang digunakan: cuda
Nama GPU: Tesla T4
```

4. Load CIFAR-10

Dataset dimuat langsung menggunakan `torchvision.datasets.CIFAR10`. Saat pertama kali dijalankan, dataset (~170MB) akan diunduh otomatis. `train=True` memberi 50.000 gambar training, `train=False` memberi 10.000 gambar test. Kita muat dulu tanpa transform, transform ditentukan kemudian di bagian Preprocessing.

```
DATA_DIR = "./data"

# Unduh dataset mentah (belum ditransformasi) hanya untuk keperluan
EDA
raw_train = torchvision.datasets.CIFAR10(root=DATA_DIR, train=True,
download=True)
raw_test = torchvision.datasets.CIFAR10(root=DATA_DIR, train=False,
```

```
download=True)

CLASSES = ['airplane', 'automobile', 'bird', 'cat', 'deer',
           'dog', 'frog', 'horse', 'ship', 'truck']

print("Jumlah data training :", len(raw_train))
print("Jumlah data test     :", len(raw_test))
print("Kelas:", CLASSES)
```

```
100%|██████████| 170M/170M [43:29<00:00, 65.3kB/s]

Jumlah data training : 50000
Jumlah data test     : 10000
Kelas: ['airplane', 'automobile', 'bird', 'cat', 'deer', 'dog',
'frog', 'horse', 'ship', 'truck']
```

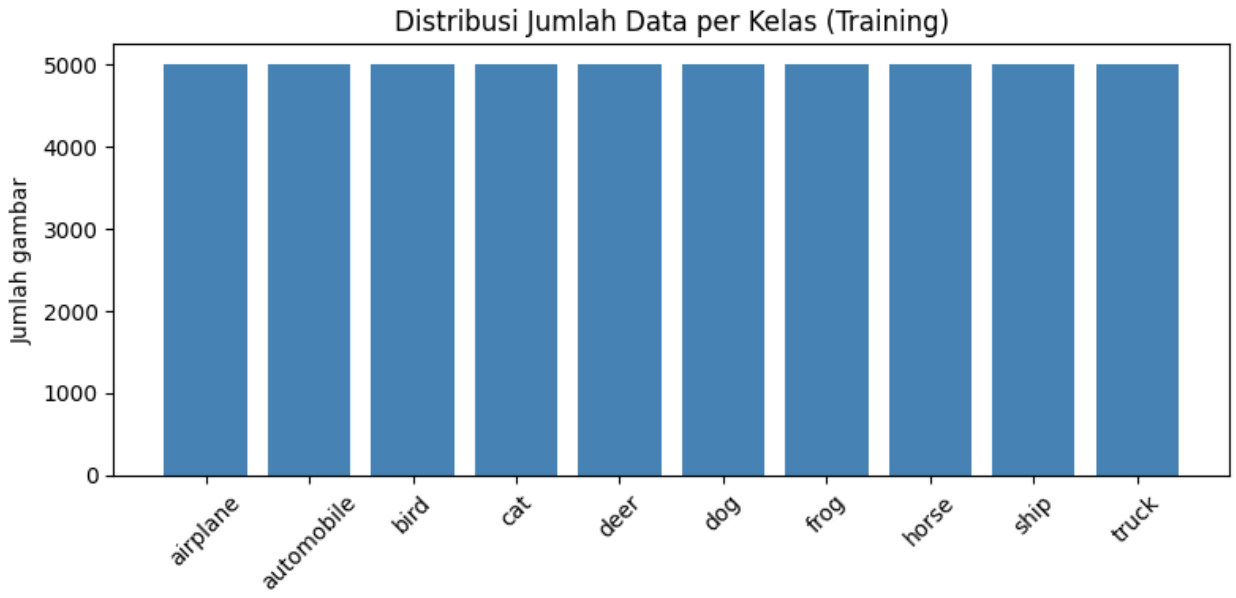
5. Exploratory Data Analysis (EDA) Sederhana

Melihat contoh gambar dan distribusi jumlah data per kelas, untuk memastikan dataset seimbang (balanced).

```
# Distribusi kelas pada data training
labels_train = np.array(raw_train.targets)
unique, counts = np.unique(labels_train, return_counts=True)

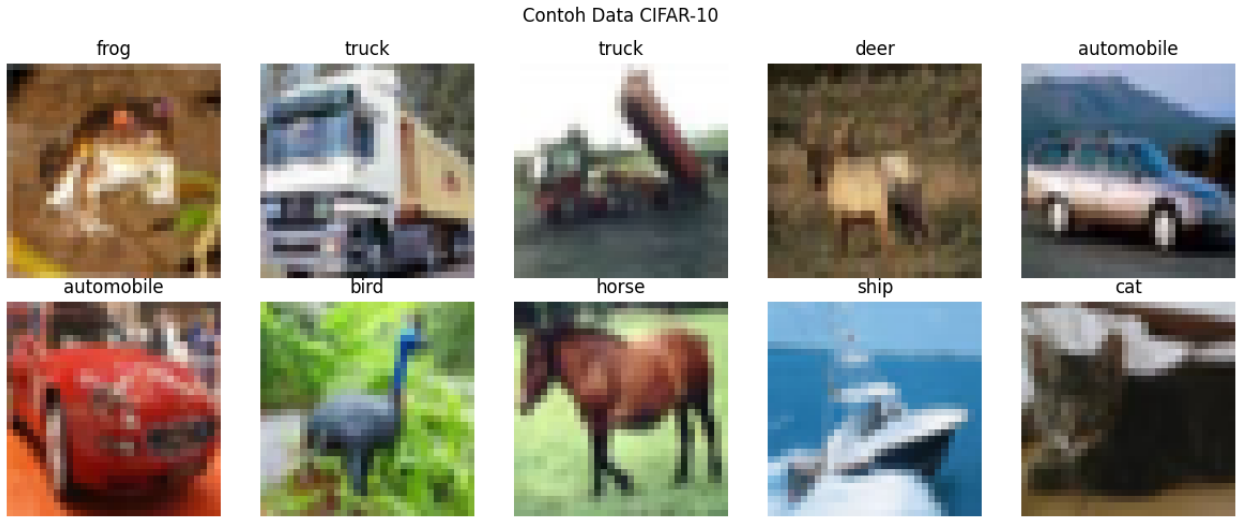
plt.figure(figsize=(8,4))
plt.bar([CLASSES[i] for i in unique], counts, color="steelblue")
plt.title("Distribusi Jumlah Data per Kelas (Training)")
plt.ylabel("Jumlah gambar")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

print("Ukuran gambar (H,W,C):", np.array(raw_train.data[0]).shape)
```




```
Ukuran gambar (H,W,C): (32, 32, 3)
```

```
# Menampilkan beberapa contoh gambar
fig, axes = plt.subplots(2, 5, figsize=(12,5))
for i, ax in enumerate(axes.flatten()):
    img, label = raw_train[i]
    ax.imshow(img)
    ax.set_title(CLASSES[label])
    ax.axis("off")
plt.suptitle("Contoh Data CIFAR-10")
plt.tight_layout()
plt.show()
```



6. Preprocessing & 7. Data Augmentation

Karena model adalah ANN/MLP (bukan CNN), gambar tetap dibaca sebagai tensor 3x32x32 pada tahap transform (flatten dilakukan nanti di dalam model, lihat bagian *Baseline Model*).

Untuk **data training** kita gunakan augmentasi dasar yang wajar untuk CIFAR-10: `RandomHorizontalFlip` dan `RandomCrop` — lalu `ToTensor` dan `Normalize`.

Untuk **data validation & test**, TIDAK ada augmentasi random (hanya `ToTensor` + `Normalize`), agar evaluasi konsisten dan adil.

Nilai mean/std yang dipakai adalah nilai standar untuk CIFAR-10.

```
CIFAR_MEAN = (0.4914, 0.4822, 0.4465)
CIFAR_STD  = (0.2470, 0.2435, 0.2616)

train_transform = transforms.Compose([
    transforms.RandomCrop(32, padding=4),   # augmentasi: crop acak
dengan padding
    transforms.RandomHorizontalFlip(),      # augmentasi: flip
horizontal acak
    transforms.ToTensor(),
    transforms.Normalize(CIFAR_MEAN, CIFAR_STD),
])

eval_transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize(CIFAR_MEAN, CIFAR_STD),
])
```

8. Train-Validation-Test Split

Data training asli (50.000) dibagi menjadi **45.000 training** dan **5.000 validation** secara acak (dengan seed tetap agar reproducible). Data test (10.000) TIDAK disentuh sampai evaluasi akhir — tidak digunakan untuk memilih model terbaik, sesuai aturan tugas.

Trik penting: karena transform training (dengan augmentasi) dan validation (tanpa augmentasi) harus berbeda, kita membuat dua objek dataset CIFAR10 terpisah dengan transform berbeda, lalu memakai indeks split yang SAMA (dari `random_split` pada objek dummy) agar pembagian data konsisten.

```
full_train_aug  = torchvision.datasets.CIFAR10(root=DATA_DIR,
train=True, download=False, transform=train_transform)
full_train_eval = torchvision.datasets.CIFAR10(root=DATA_DIR,
train=True, download=False, transform=eval_transform)
test_set        = torchvision.datasets.CIFAR10(root=DATA_DIR,
train=False, download=False, transform=eval_transform)

N_TRAIN = 45000
```

```
N_VAL   = 5000

generator = torch.Generator().manual_seed(SEED)
train_indices, val_indices = random_split(range(len(full_train_aug)),
[N_TRAIN, N_VAL], generator=generator)

train_set = torch.utils.data.Subset(full_train_aug,
train_indices.indices)   # pakai augmentasi
val_set   = torch.utils.data.Subset(full_train_eval,
val_indices.indices)     # tanpa augmentasi

print("Jumlah data training  :", len(train_set))
print("Jumlah data validation:", len(val_set))
print("Jumlah data test      :", len(test_set))
```

```
Jumlah data training  : 45000
Jumlah data validation: 5000
Jumlah data test      : 10000
```

9. DataLoader

Membungkus dataset ke dalam `DataLoader` agar data dapat diambil per batch selama training.

```
BATCH_SIZE = 128

train_loader = DataLoader(train_set, batch_size=BATCH_SIZE,
shuffle=True,  num_workers=2, pin_memory=True)
val_loader   = DataLoader(val_set,   batch_size=BATCH_SIZE,
shuffle=False, num_workers=2, pin_memory=True)
test_loader  = DataLoader(test_set,  batch_size=BATCH_SIZE,
shuffle=False, num_workers=2, pin_memory=True)

print("Jumlah batch training  :", len(train_loader))
print("Jumlah batch validation:", len(val_loader))
print("Jumlah batch test      :", len(test_loader))
```

```
Jumlah batch training  : 352
Jumlah batch validation: 40
Jumlah batch test      : 79
```

10. Visualisasi Contoh Data (Setelah Augmentasi)

Memastikan augmentasi bekerja: gambar yang sama bisa terlihat berbeda (flip/crop) setiap kali diambil.

```
6def denormalize(img_tensor):
    img = img_tensor.clone()
    for t, m, s in zip(img, CIFAR_MEAN, CIFAR_STD):
        t.mul_(s).add_(m)
```

```
    return img.permute(1,2,0).numpy().clip(0,1)

images, labels = next(iter(train_loader))
fig, axes = plt.subplots(1, 5, figsize=(12,3))
for i, ax in enumerate(axes):
    ax.imshow(denormalize(images[i]))
    ax.set_title(CLASSES[labels[i]])
    ax.axis("off")
plt.suptitle("Contoh Batch Training (dengan augmentasi)")
plt.tight_layout()
plt.show()
```



11. Arsitektur ANN (Baseline & Fleksibel untuk Eksperimen)

Karena model adalah ANN/MLP, gambar berukuran `[batch, 3, 32, 32]` di-**flatten** menjadi `[batch, 3072]` sebelum masuk ke fully-connected layer pertama — dilakukan dengan `x = x.view(x.size(0), -1)`. Flattening diperlukan karena layer `nn.Linear` hanya menerima input berbentuk vektor 1 dimensi per sampel, bukan tensor spasial 3 dimensi seperti pada CNN.

Kelas `MLP` dibuat fleksibel (menerima daftar hidden layer, opsi dropout, dan opsi BatchNorm) supaya bisa dipakai ulang untuk seluruh eksperimen (kapasitas model, regularisasi, dst) tanpa menulis ulang arsitektur setiap kali.

Baseline: 3072 → 512 → 256 → 10, tanpa dropout, tanpa BatchNorm.

```
class MLP(nn.Module):
    def __init__(self, input_dim=3072, hidden_layers=(512,256),
num_classes=10,
                 dropout=0.0, use_bn=False):
        super().__init__()
        layers = []
        in_dim = input_dim
        for h in hidden_layers:
            layers.append(nn.Linear(in_dim, h))
            if use_bn:
                layers.append(nn.BatchNorm1d(h))   # BatchNorm sebelum
aktivasi
            layers.append(nn.ReLU())
            if dropout > 0:
```

```
                layers.append(nn.Dropout(dropout)) # Dropout setelah
aktivasi
            in_dim = h
        layers.append(nn.Linear(in_dim, num_classes))
        self.net = nn.Sequential(*layers)

    def forward(self, x):
        x = x.view(x.size(0), -1)   # flatten: [B,3,32,32] -> [B,3072]
        return self.net(x)

def count_params(model):
    return sum(p.numel() for p in model.parameters() if
p.requires_grad)

baseline_model = MLP(hidden_layers=(512,256), dropout=0.0,
use_bn=False).to(device)
print(baseline_model)
print("Jumlah parameter:", count_params(baseline_model))
```

```
MLP(
  (net): Sequential(
    (0): Linear(in_features=3072, out_features=512, bias=True)
    (1): ReLU()
    (2): Linear(in_features=512, out_features=256, bias=True)
    (3): ReLU()
    (4): Linear(in_features=256, out_features=10, bias=True)
  )
)
Jumlah parameter: 1707274
```

12. Early Stopping (Implementasi Manual)

Early stopping dibuat sendiri (bukan library eksternal). Cara kerjanya:

- Setiap epoch, jika `validation loss` **membaik** (lebih kecil dari nilai terbaik sebelumnya), simpan bobot model sebagai checkpoint terbaik dan reset counter `patience`.
- Jika `validation loss` **tidak membaik**, counter bertambah 1.
- Jika counter mencapai nilai `patience` (misal 5), training dihentikan lebih awal karena model dianggap sudah berhenti belajar / mulai overfitting.

```
class EarlyStopping:
    def __init__(self, patience=5, checkpoint_path="best_model.pt"):
        self.patience = patience
        self.checkpoint_path = checkpoint_path
        self.best_loss = float("inf")
        self.counter = 0
        self.best_epoch = 0
        self.should_stop = False
```

```
    def step(self, val_loss, model, epoch):
        if val_loss < self.best_loss:
            self.best_loss = val_loss
            self.counter = 0
            self.best_epoch = epoch
            torch.save(model.state_dict(), self.checkpoint_path)  # 
simpan checkpoint terbaik
        else:
            self.counter += 1
            if self.counter >= self.patience:
                self.should_stop = True
```

13. Fungsi Training & Validation

Satu fungsi generik `run_training` dipakai untuk semua eksperimen agar perbandingan konsisten (controlled experiment). Fungsi ini mencatat `train_loss`, `val_loss`, `train_accuracy`, `val_accuracy` setiap epoch, menerapkan early stopping + checkpointing, dan mengukur `training_time`.

```
def train_one_epoch(model, loader, criterion, optimizer):
    model.train()
    total_loss, correct, total = 0.0, 0, 0
    for x, y in loader:
        x, y = x.to(device), y.to(device)
        optimizer.zero_grad()
        out = model(x)
        loss = criterion(out, y)
        loss.backward()
        optimizer.step()

        total_loss += loss.item() * x.size(0)
        correct += (out.argmax(1) == y).sum().item()
        total += x.size(0)
    return total_loss/total, correct/total

@torch.no_grad()
def evaluate(model, loader, criterion):
    model.eval()
    total_loss, correct, total = 0.0, 0, 0
    for x, y in loader:
        x, y = x.to(device), y.to(device)
        out = model(x)
        loss = criterion(out, y)
        total_loss += loss.item() * x.size(0)
        correct += (out.argmax(1) == y).sum().item()
        total += x.size(0)
    return total_loss/total, correct/total
```

```
def run_training(model, train_loader, val_loader, optimizer,
scheduler=None,
                 epochs=30, patience=5, exp_name="exp",
verbose=True):
    criterion = nn.CrossEntropyLoss()
    early_stopper = EarlyStopping(patience=patience,
checkpoint_path=f"{exp_name}_best.pt")
    history = {"train_loss": [], "val_loss": [], "train_acc": [],
"val_acc": []}

    start_time = time.time()
    for epoch in range(1, epochs+1):
        tr_loss, tr_acc = train_one_epoch(model, train_loader,
criterion, optimizer)
        val_loss, val_acc = evaluate(model, val_loader, criterion)

        if scheduler is not None:
            if isinstance(scheduler,
optim.lr_scheduler.ReduceLROnPlateau):
                scheduler.step(val_loss)
            else:
                scheduler.step()

        history["train_loss"].append(tr_loss)
        history["val_loss"].append(val_loss)
        history["train_acc"].append(tr_acc)
        history["val_acc"].append(val_acc)

        if verbose:
            print(f"[{exp_name}] Epoch {epoch:02d} |
train_loss={tr_loss:.4f} acc={tr_acc:.4f} "
                  f"| val_loss={val_loss:.4f} acc={val_acc:.4f}")

        early_stopper.step(val_loss, model, epoch)
        if early_stopper.should_stop:
            if verbose:
                print(f"Early stopping dipicu pada epoch {epoch}
(patience={patience})")
            break

    training_time = time.time() - start_time

    # load kembali bobot terbaik sebelum dikembalikan
    model.load_state_dict(torch.load(f"{exp_name}_best.pt"))

    result = {
        "exp_name": exp_name,
        "best_epoch": early_stopper.best_epoch,
        "best_val_loss": early_stopper.best_loss,
        "best_val_acc": max(history["val_acc"]),
```

```
        "training_time": training_time,
        "epochs_run": len(history["train_loss"]),
    }
    return model, history, result
```

14. Eksperimen 1 — Baseline

Baseline: arsitektur 3072→512→256→10, optimizer Adam (lr=0.001), tanpa dropout, tanpa BatchNorm, tanpa scheduler, epoch maksimal 30 dengan early stopping (patience=5).

Jalankan sel ini di Colab. Hasil (loss, akurasi, waktu training, epoch berhenti) akan muncul sebagai output asli — jangan diedit manual.

```
set_seed(SEED)
model_baseline = MLP(hidden_layers=(512,256), dropout=0.0,
use_bn=False).to(device)
opt_baseline = optim.Adam(model_baseline.parameters(), lr=0.001)

model_baseline, hist_baseline, res_baseline = run_training(
    model_baseline, train_loader, val_loader, opt_baseline,
    scheduler=None, epochs=30, patience=5, exp_name="baseline"
)

print("\nRingkasan Baseline:", res_baseline)
```

```
[baseline] Epoch 01 | train_loss=1.8068 acc=0.3473 | val_loss=1.7485
acc=0.3948
[baseline] Epoch 02 | train_loss=1.6379 acc=0.4112 | val_loss=1.6502
acc=0.4136
[baseline] Epoch 03 | train_loss=1.5688 acc=0.4348 | val_loss=1.6467
acc=0.4096
[baseline] Epoch 04 | train_loss=1.5299 acc=0.4503 | val_loss=1.5933
acc=0.4320
[baseline] Epoch 05 | train_loss=1.5014 acc=0.4622 | val_loss=1.5750
acc=0.4334
[baseline] Epoch 06 | train_loss=1.4733 acc=0.4690 | val_loss=1.5151
acc=0.4632
[baseline] Epoch 07 | train_loss=1.4557 acc=0.4732 | val_loss=1.5485
acc=0.4564
[baseline] Epoch 08 | train_loss=1.4354 acc=0.4829 | val_loss=1.5727
acc=0.4452
[baseline] Epoch 09 | train_loss=1.4177 acc=0.4924 | val_loss=1.5344
acc=0.4692
[baseline] Epoch 10 | train_loss=1.4054 acc=0.4957 | val_loss=1.5721
acc=0.4540
[baseline] Epoch 11 | train_loss=1.3918 acc=0.5011 | val_loss=1.5609
acc=0.4506
Early stopping dipicu pada epoch 11 (patience=5)
```

```
Ringkasan Baseline: {'exp_name': 'baseline', 'best_epoch': 6,
'best_val_loss': 1.5151244644165038, 'best_val_acc': 0.4692,
'training_time': 179.3498923778534, 'epochs_run': 11}
```

15. Eksperimen 2 — Optimizer (Adam vs SGD+Momentum)

Arsitektur, epoch, batch size, dan regularisasi dibuat SAMA dengan baseline (controlled experiment) — hanya optimizer yang diubah, agar perbandingan adil.

- A. Adam (lr=0.001) — sudah dijalankan sebagai baseline di atas, akan dipakai ulang hasilnya.
- B. SGD + Momentum (lr=0.01, momentum=0.9)

```
set_seed(SEED)
model_sgd = MLP(hidden_layers=(512,256), dropout=0.0,
use_bn=False).to(device)
opt_sgd = optim.SGD(model_sgd.parameters(), lr=0.01, momentum=0.9)

model_sgd, hist_sgd, res_sgd = run_training(
    model_sgd, train_loader, val_loader, opt_sgd,
    scheduler=None, epochs=30, patience=5, exp_name="sgd_momentum"
)

print("\nRingkasan SGD+Momentum:", res_sgd)
```

```
[sgd_momentum] Epoch 01 | train_loss=1.8556 acc=0.3289 |
val_loss=1.7315 acc=0.3858
[sgd_momentum] Epoch 02 | train_loss=1.6617 acc=0.4036 |
val_loss=1.6148 acc=0.4322
[sgd_momentum] Epoch 03 | train_loss=1.5823 acc=0.4342 |
val_loss=1.5755 acc=0.4436
[sgd_momentum] Epoch 04 | train_loss=1.5326 acc=0.4514 |
val_loss=1.5287 acc=0.4642
[sgd_momentum] Epoch 05 | train_loss=1.5008 acc=0.4617 |
val_loss=1.5094 acc=0.4726
[sgd_momentum] Epoch 06 | train_loss=1.4724 acc=0.4713 |
val_loss=1.4484 acc=0.4906
[sgd_momentum] Epoch 07 | train_loss=1.4486 acc=0.4788 |
val_loss=1.4601 acc=0.4872
[sgd_momentum] Epoch 08 | train_loss=1.4243 acc=0.4891 |
val_loss=1.4705 acc=0.4844
[sgd_momentum] Epoch 09 | train_loss=1.4021 acc=0.4983 |
val_loss=1.4444 acc=0.4954
[sgd_momentum] Epoch 10 | train_loss=1.3857 acc=0.5002 |
val_loss=1.4471 acc=0.4862
[sgd_momentum] Epoch 11 | train_loss=1.3722 acc=0.5084 |
val_loss=1.4345 acc=0.5052
[sgd_momentum] Epoch 12 | train_loss=1.3577 acc=0.5138 |
val_loss=1.3963 acc=0.5080
[sgd_momentum] Epoch 13 | train_loss=1.3467 acc=0.5176 |
```

```
val_loss=1.3852 acc=0.5124
[sgd_momentum] Epoch 14 | train_loss=1.3358 acc=0.5211 |
val_loss=1.3802 acc=0.5172
[sgd_momentum] Epoch 15 | train_loss=1.3206 acc=0.5276 |
val_loss=1.4056 acc=0.5080
[sgd_momentum] Epoch 16 | train_loss=1.3134 acc=0.5294 |
val_loss=1.3892 acc=0.5118
[sgd_momentum] Epoch 17 | train_loss=1.3050 acc=0.5322 |
val_loss=1.4033 acc=0.5038
[sgd_momentum] Epoch 18 | train_loss=1.2937 acc=0.5377 |
val_loss=1.4032 acc=0.5072
[sgd_momentum] Epoch 19 | train_loss=1.2909 acc=0.5358 |
val_loss=1.3804 acc=0.5062
Early stopping dipicu pada epoch 19 (patience=5)

Ringkasan SGD+Momentum: {'exp_name': 'sgd_momentum', 'best_epoch': 14,
'best_val_loss': 1.380197254562378, 'best_val_acc': 0.5172,
'training_time': 309.85204100608826, 'epochs_run': 19}
```

16. Eksperimen 3 — Scheduler (Tanpa vs StepLR)

Komponen lain dibuat sama dengan baseline. Kita bandingkan training TANPA scheduler (baseline di atas) VS DENGAN `StepLR` (learning rate dikalikan `gamma` setiap `step_size` epoch).

```
set_seed(SEED)
model_step = MLP(hidden_layers=(512,256), dropout=0.0,
use_bn=False).to(device)
opt_step = optim.Adam(model_step.parameters(), lr=0.001)
sched_step = optim.lr_scheduler.StepLR(opt_step, step_size=7,
gamma=0.5)

model_step, hist_step, res_step = run_training(
    model_step, train_loader, val_loader, opt_step,
    scheduler=sched_step, epochs=30, patience=5, exp_name="steplr"
)

print("\nRingkasan dengan StepLR:", res_step)
```

```
[steplr] Epoch 01 | train_loss=1.8068 acc=0.3473 | val_loss=1.7485
acc=0.3948
[steplr] Epoch 02 | train_loss=1.6379 acc=0.4112 | val_loss=1.6502
acc=0.4136
[steplr] Epoch 03 | train_loss=1.5688 acc=0.4348 | val_loss=1.6467
acc=0.4096
[steplr] Epoch 04 | train_loss=1.5299 acc=0.4503 | val_loss=1.5933
acc=0.4320
[steplr] Epoch 05 | train_loss=1.5014 acc=0.4622 | val_loss=1.5750
acc=0.4334
[steplr] Epoch 06 | train_loss=1.4733 acc=0.4690 | val_loss=1.5151
```

```
acc=0.4632
[steplr] Epoch 07 | train_loss=1.4557 acc=0.4732 | val_loss=1.5485
acc=0.4564
[steplr] Epoch 08 | train_loss=1.3852 acc=0.5039 | val_loss=1.4688
acc=0.4830
[steplr] Epoch 09 | train_loss=1.3580 acc=0.5125 | val_loss=1.4634
acc=0.4876
[steplr] Epoch 10 | train_loss=1.3453 acc=0.5162 | val_loss=1.4803
acc=0.4836
[steplr] Epoch 11 | train_loss=1.3328 acc=0.5186 | val_loss=1.4595
acc=0.4930
[steplr] Epoch 12 | train_loss=1.3219 acc=0.5243 | val_loss=1.4249
acc=0.5046
[steplr] Epoch 13 | train_loss=1.3124 acc=0.5290 | val_loss=1.4221
acc=0.5016
[steplr] Epoch 14 | train_loss=1.3022 acc=0.5335 | val_loss=1.4220
acc=0.5068
[steplr] Epoch 15 | train_loss=1.2600 acc=0.5478 | val_loss=1.4175
acc=0.5070
[steplr] Epoch 16 | train_loss=1.2492 acc=0.5528 | val_loss=1.4005
acc=0.5176
[steplr] Epoch 17 | train_loss=1.2416 acc=0.5561 | val_loss=1.4029
acc=0.5152
[steplr] Epoch 18 | train_loss=1.2340 acc=0.5587 | val_loss=1.4099
acc=0.5058
[steplr] Epoch 19 | train_loss=1.2311 acc=0.5580 | val_loss=1.3755
acc=0.5186
[steplr] Epoch 20 | train_loss=1.2209 acc=0.5616 | val_loss=1.4056
acc=0.5110
[steplr] Epoch 21 | train_loss=1.2160 acc=0.5629 | val_loss=1.4110
acc=0.5098
[steplr] Epoch 22 | train_loss=1.1932 acc=0.5702 | val_loss=1.3781
acc=0.5272
[steplr] Epoch 23 | train_loss=1.1859 acc=0.5719 | val_loss=1.3671
acc=0.5240
[steplr] Epoch 24 | train_loss=1.1821 acc=0.5788 | val_loss=1.3710
acc=0.5262
[steplr] Epoch 25 | train_loss=1.1820 acc=0.5728 | val_loss=1.3593
acc=0.5336
[steplr] Epoch 26 | train_loss=1.1759 acc=0.5792 | val_loss=1.3635
acc=0.5314
[steplr] Epoch 27 | train_loss=1.1742 acc=0.5783 | val_loss=1.3591
acc=0.5348
[steplr] Epoch 28 | train_loss=1.1691 acc=0.5795 | val_loss=1.3621
acc=0.5320
[steplr] Epoch 29 | train_loss=1.1584 acc=0.5856 | val_loss=1.3447
acc=0.5366
[steplr] Epoch 30 | train_loss=1.1530 acc=0.5862 | val_loss=1.3383
acc=0.5350
```

```
Ringkasan dengan StepLR: {'exp_name': 'steplr', 'best_epoch': 30,
'best_val_loss': 1.338266152381897, 'best_val_acc': 0.5366,
'training_time': 486.66291856765747, 'epochs_run': 30}
```

17. Eksperimen 4 — Regularisasi (Dropout & Batch Normalization)

Empat kombinasi dijalankan dengan optimizer, scheduler, dan arsitektur yang SAMA (hanya dropout/BatchNorm yang diubah):

1. Baseline (sudah dijalankan di atas)
2. Baseline + Dropout (p=0.3)
3. Baseline + BatchNorm
4. Baseline + Dropout + BatchNorm

Urutan layer: `Linear -> BatchNorm -> ReLU -> Dropout`, urutan yang umum digunakan agar BatchNorm menormalkan output linear sebelum aktivasi, dan Dropout diterapkan setelah aktivasi.

```
set_seed(SEED)
model_dropout = MLP(hidden_layers=(512,256), dropout=0.3,
use_bn=False).to(device)
opt_dropout = optim.Adam(model_dropout.parameters(), lr=0.001,
weight_decay=1e-4)
model_dropout, hist_dropout, res_dropout = run_training(
    model_dropout, train_loader, val_loader, opt_dropout,
    scheduler=None, epochs=30, patience=5, exp_name="dropout"
)
print("\nRingkasan Baseline+Dropout:", res_dropout)
```

```
[dropout] Epoch 01 | train_loss=1.9139 acc=0.3067 | val_loss=1.7913
acc=0.3632
[dropout] Epoch 02 | train_loss=1.7966 acc=0.3529 | val_loss=1.7259
acc=0.3884
[dropout] Epoch 03 | train_loss=1.7452 acc=0.3721 | val_loss=1.6810
acc=0.4028
[dropout] Epoch 04 | train_loss=1.7132 acc=0.3831 | val_loss=1.6966
acc=0.4146
[dropout] Epoch 05 | train_loss=1.6971 acc=0.3882 | val_loss=1.6741
acc=0.4144
[dropout] Epoch 06 | train_loss=1.6816 acc=0.3932 | val_loss=1.6593
acc=0.4172
[dropout] Epoch 07 | train_loss=1.6729 acc=0.3986 | val_loss=1.6622
acc=0.4040
[dropout] Epoch 08 | train_loss=1.6638 acc=0.4024 | val_loss=1.6654
acc=0.4114
[dropout] Epoch 09 | train_loss=1.6518 acc=0.4058 | val_loss=1.6719
```

```
acc=0.4152
[dropout] Epoch 10 | train_loss=1.6487 acc=0.4044 | val_loss=1.6179
acc=0.4198
[dropout] Epoch 11 | train_loss=1.6430 acc=0.4085 | val_loss=1.6462
acc=0.4224
[dropout] Epoch 12 | train_loss=1.6327 acc=0.4120 | val_loss=1.6350
acc=0.4316
[dropout] Epoch 13 | train_loss=1.6305 acc=0.4138 | val_loss=1.6046
acc=0.4352
[dropout] Epoch 14 | train_loss=1.6296 acc=0.4154 | val_loss=1.6284
acc=0.4314
[dropout] Epoch 15 | train_loss=1.6214 acc=0.4204 | val_loss=1.5982
acc=0.4496
[dropout] Epoch 16 | train_loss=1.6169 acc=0.4226 | val_loss=1.6188
acc=0.4388
[dropout] Epoch 17 | train_loss=1.6207 acc=0.4190 | val_loss=1.5932
acc=0.4432
[dropout] Epoch 18 | train_loss=1.6128 acc=0.4205 | val_loss=1.6101
acc=0.4272
[dropout] Epoch 19 | train_loss=1.6136 acc=0.4201 | val_loss=1.5660
acc=0.4476
[dropout] Epoch 20 | train_loss=1.6044 acc=0.4264 | val_loss=1.5979
acc=0.4316
[dropout] Epoch 21 | train_loss=1.6032 acc=0.4256 | val_loss=1.5946
acc=0.4344
[dropout] Epoch 22 | train_loss=1.6031 acc=0.4222 | val_loss=1.5746
acc=0.4408
[dropout] Epoch 23 | train_loss=1.6006 acc=0.4235 | val_loss=1.5624
acc=0.4410
[dropout] Epoch 24 | train_loss=1.6010 acc=0.4240 | val_loss=1.5961
acc=0.4448
[dropout] Epoch 25 | train_loss=1.6039 acc=0.4259 | val_loss=1.6053
acc=0.4296
[dropout] Epoch 26 | train_loss=1.5889 acc=0.4288 | val_loss=1.6111
acc=0.4356
[dropout] Epoch 27 | train_loss=1.5932 acc=0.4272 | val_loss=1.5796
acc=0.4488
[dropout] Epoch 28 | train_loss=1.5958 acc=0.4290 | val_loss=1.6050
acc=0.4214
Early stopping dipicu pada epoch 28 (patience=5)

Ringkasan Baseline+Dropout: {'exp_name': 'dropout', 'best_epoch': 23,
'best_val_loss': 1.5624053495407104, 'best_val_acc': 0.4496,
'training_time': 466.28743982315063, 'epochs_run': 28}

set_seed(SEED)
model_bn = MLP(hidden_layers=(512,256), dropout=0.0,
use_bn=True).to(device)
opt_bn = optim.Adam(model_bn.parameters(), lr=0.001, weight_decay=1e-
4)
```

```
model_bn, hist_bn, res_bn = run_training(
    model_bn, train_loader, val_loader, opt_bn,
    scheduler=None, epochs=30, patience=5, exp_name="batchnorm"
)
print("\nRingkasan Baseline+BatchNorm:", res_bn)
```

```
[batchnorm] Epoch 01 | train_loss=1.7723 acc=0.3571 | val_loss=1.6018
acc=0.4250
[batchnorm] Epoch 02 | train_loss=1.5990 acc=0.4216 | val_loss=1.4873
acc=0.4740
[batchnorm] Epoch 03 | train_loss=1.5352 acc=0.4462 | val_loss=1.4484
acc=0.4804
[batchnorm] Epoch 04 | train_loss=1.4937 acc=0.4607 | val_loss=1.4247
acc=0.4864
[batchnorm] Epoch 05 | train_loss=1.4650 acc=0.4727 | val_loss=1.3901
acc=0.5102
[batchnorm] Epoch 06 | train_loss=1.4437 acc=0.4764 | val_loss=1.3402
acc=0.5196
[batchnorm] Epoch 07 | train_loss=1.4250 acc=0.4808 | val_loss=1.3714
acc=0.5062
[batchnorm] Epoch 08 | train_loss=1.4055 acc=0.4936 | val_loss=1.3584
acc=0.5124
[batchnorm] Epoch 09 | train_loss=1.3920 acc=0.5001 | val_loss=1.3191
acc=0.5268
[batchnorm] Epoch 10 | train_loss=1.3788 acc=0.5054 | val_loss=1.3163
acc=0.5270
[batchnorm] Epoch 11 | train_loss=1.3652 acc=0.5091 | val_loss=1.2881
acc=0.5380
[batchnorm] Epoch 12 | train_loss=1.3563 acc=0.5134 | val_loss=1.2978
acc=0.5382
[batchnorm] Epoch 13 | train_loss=1.3491 acc=0.5143 | val_loss=1.2722
acc=0.5492
[batchnorm] Epoch 14 | train_loss=1.3381 acc=0.5203 | val_loss=1.2811
acc=0.5432
[batchnorm] Epoch 15 | train_loss=1.3231 acc=0.5261 | val_loss=1.2738
acc=0.5488
[batchnorm] Epoch 16 | train_loss=1.3218 acc=0.5262 | val_loss=1.2564
acc=0.5544
[batchnorm] Epoch 17 | train_loss=1.3138 acc=0.5302 | val_loss=1.2545
acc=0.5514
[batchnorm] Epoch 18 | train_loss=1.3088 acc=0.5282 | val_loss=1.2513
acc=0.5580
[batchnorm] Epoch 19 | train_loss=1.3046 acc=0.5320 | val_loss=1.2479
acc=0.5488
[batchnorm] Epoch 20 | train_loss=1.2910 acc=0.5353 | val_loss=1.2663
acc=0.5536
[batchnorm] Epoch 21 | train_loss=1.2884 acc=0.5373 | val_loss=1.2515
acc=0.5536
[batchnorm] Epoch 22 | train_loss=1.2800 acc=0.5414 | val_loss=1.2415
acc=0.5580
```

```
[batchnorm] Epoch 23 | train_loss=1.2752 acc=0.5426 | val_loss=1.2315
acc=0.5630
[batchnorm] Epoch 24 | train_loss=1.2709 acc=0.5446 | val_loss=1.2304
acc=0.5626
[batchnorm] Epoch 25 | train_loss=1.2671 acc=0.5450 | val_loss=1.2142
acc=0.5692
[batchnorm] Epoch 26 | train_loss=1.2585 acc=0.5491 | val_loss=1.2216
acc=0.5690
[batchnorm] Epoch 27 | train_loss=1.2561 acc=0.5488 | val_loss=1.2234
acc=0.5774
[batchnorm] Epoch 28 | train_loss=1.2471 acc=0.5518 | val_loss=1.2132
acc=0.5732
[batchnorm] Epoch 29 | train_loss=1.2485 acc=0.5510 | val_loss=1.2198
acc=0.5706
[batchnorm] Epoch 30 | train_loss=1.2458 acc=0.5529 | val_loss=1.2012
acc=0.5790

Ringkasan Baseline+BatchNorm: {'exp_name': 'batchnorm', 'best_epoch':
30, 'best_val_loss': 1.2011934938430786, 'best_val_acc': 0.579,
'training_time': 512.6446645259857, 'epochs_run': 30}
```

```
set_seed(SEED)
model_dropbn = MLP(hidden_layers=(512,256), dropout=0.3,
use_bn=True).to(device)
opt_dropbn = optim.Adam(model_dropbn.parameters(), lr=0.001,
weight_decay=1e-4)
model_dropbn, hist_dropbn, res_dropbn = run_training(
    model_dropbn, train_loader, val_loader, opt_dropbn,
    scheduler=None, epochs=30, patience=5,
exp_name="dropout_batchnorm"
)
print("\nRingkasan Baseline+Dropout+BatchNorm:", res_dropbn)
```

```
[dropout_batchnorm] Epoch 01 | train_loss=1.8560 acc=0.3217 |
val_loss=1.6485 acc=0.4152
[dropout_batchnorm] Epoch 02 | train_loss=1.7035 acc=0.3854 |
val_loss=1.5466 acc=0.4474
[dropout_batchnorm] Epoch 03 | train_loss=1.6510 acc=0.4035 |
val_loss=1.5122 acc=0.4554
[dropout_batchnorm] Epoch 04 | train_loss=1.6132 acc=0.4168 |
val_loss=1.4704 acc=0.4754
[dropout_batchnorm] Epoch 05 | train_loss=1.5894 acc=0.4265 |
val_loss=1.4426 acc=0.4826
[dropout_batchnorm] Epoch 06 | train_loss=1.5713 acc=0.4324 |
val_loss=1.4095 acc=0.4938
[dropout_batchnorm] Epoch 07 | train_loss=1.5550 acc=0.4346 |
val_loss=1.4341 acc=0.4908
[dropout_batchnorm] Epoch 08 | train_loss=1.5435 acc=0.4395 |
val_loss=1.4137 acc=0.4920
[dropout_batchnorm] Epoch 09 | train_loss=1.5336 acc=0.4447 |
```

```
val_loss=1.3822 acc=0.5044
[dropout_batchnorm] Epoch 10 | train_loss=1.5245 acc=0.4489 |
val_loss=1.3720 acc=0.5010
[dropout_batchnorm] Epoch 11 | train_loss=1.5173 acc=0.4527 |
val_loss=1.3579 acc=0.5118
[dropout_batchnorm] Epoch 12 | train_loss=1.5086 acc=0.4540 |
val_loss=1.3573 acc=0.5118
[dropout_batchnorm] Epoch 13 | train_loss=1.5043 acc=0.4586 |
val_loss=1.3497 acc=0.5196
[dropout_batchnorm] Epoch 14 | train_loss=1.5031 acc=0.4593 |
val_loss=1.3494 acc=0.5140
[dropout_batchnorm] Epoch 15 | train_loss=1.4923 acc=0.4620 |
val_loss=1.3300 acc=0.5282
[dropout_batchnorm] Epoch 16 | train_loss=1.4912 acc=0.4658 |
val_loss=1.3293 acc=0.5258
[dropout_batchnorm] Epoch 17 | train_loss=1.4895 acc=0.4625 |
val_loss=1.3400 acc=0.5268
[dropout_batchnorm] Epoch 18 | train_loss=1.4855 acc=0.4655 |
val_loss=1.3283 acc=0.5246
[dropout_batchnorm] Epoch 19 | train_loss=1.4839 acc=0.4628 |
val_loss=1.3162 acc=0.5316
[dropout_batchnorm] Epoch 20 | train_loss=1.4714 acc=0.4699 |
val_loss=1.3341 acc=0.5176
[dropout_batchnorm] Epoch 21 | train_loss=1.4730 acc=0.4696 |
val_loss=1.3315 acc=0.5192
[dropout_batchnorm] Epoch 22 | train_loss=1.4664 acc=0.4709 |
val_loss=1.3093 acc=0.5360
[dropout_batchnorm] Epoch 23 | train_loss=1.4700 acc=0.4720 |
val_loss=1.3055 acc=0.5296
[dropout_batchnorm] Epoch 24 | train_loss=1.4692 acc=0.4734 |
val_loss=1.3092 acc=0.5236
[dropout_batchnorm] Epoch 25 | train_loss=1.4671 acc=0.4710 |
val_loss=1.3092 acc=0.5362
[dropout_batchnorm] Epoch 26 | train_loss=1.4624 acc=0.4725 |
val_loss=1.3106 acc=0.5336
[dropout_batchnorm] Epoch 27 | train_loss=1.4592 acc=0.4763 |
val_loss=1.3021 acc=0.5376
[dropout_batchnorm] Epoch 28 | train_loss=1.4588 acc=0.4758 |
val_loss=1.2988 acc=0.5358
[dropout_batchnorm] Epoch 29 | train_loss=1.4578 acc=0.4755 |
val_loss=1.3046 acc=0.5358
[dropout_batchnorm] Epoch 30 | train_loss=1.4566 acc=0.4740 |
val_loss=1.2999 acc=0.5348

Ringkasan Baseline+Dropout+BatchNorm: {'exp_name':
'dropout_batchnorm', 'best_epoch': 28, 'best_val_loss':
1.2988003002166748, 'best_val_acc': 0.5376, 'training_time':
506.6082389354706, 'epochs_run': 30}
```

18. Eksperimen 5 — Kapasitas Model

Menguji tiga ukuran arsitektur dengan optimizer, scheduler, dan regularisasi yang sama:

- Kecil: 3072 → 256 → 128 → 10
- Baseline: 3072 → 512 → 256 → 10 (sudah dijalankan)
- Besar: 3072 → 1024 → 512 → 256 → 10

```
set_seed(SEED)
model_small = MLP(hidden_layers=(256,128), dropout=0.0,
use_bn=False).to(device)
opt_small = optim.Adam(model_small.parameters(), lr=0.001)
model_small, hist_small, res_small = run_training(
    model_small, train_loader, val_loader, opt_small,
    scheduler=None, epochs=30, patience=5, exp_name="small_capacity"
)
res_small["num_params"] = count_params(model_small)
print("\nRingkasan Model Kecil:", res_small)
```

```
[small_capacity] Epoch 01 | train_loss=1.8176 acc=0.3418 |
val_loss=1.7218 acc=0.3822
[small_capacity] Epoch 02 | train_loss=1.6570 acc=0.4027 |
val_loss=1.6166 acc=0.4274
[small_capacity] Epoch 03 | train_loss=1.5936 acc=0.4265 |
val_loss=1.5600 acc=0.4464
[small_capacity] Epoch 04 | train_loss=1.5451 acc=0.4443 |
val_loss=1.5560 acc=0.4450
[small_capacity] Epoch 05 | train_loss=1.5180 acc=0.4557 |
val_loss=1.5466 acc=0.4562
[small_capacity] Epoch 06 | train_loss=1.4894 acc=0.4664 |
val_loss=1.5340 acc=0.4620
[small_capacity] Epoch 07 | train_loss=1.4737 acc=0.4694 |
val_loss=1.5056 acc=0.4748
[small_capacity] Epoch 08 | train_loss=1.4507 acc=0.4780 |
val_loss=1.5161 acc=0.4662
[small_capacity] Epoch 09 | train_loss=1.4362 acc=0.4856 |
val_loss=1.4917 acc=0.4756
[small_capacity] Epoch 10 | train_loss=1.4228 acc=0.4872 |
val_loss=1.4985 acc=0.4710
[small_capacity] Epoch 11 | train_loss=1.4123 acc=0.4900 |
val_loss=1.4796 acc=0.4728
[small_capacity] Epoch 12 | train_loss=1.3992 acc=0.4962 |
val_loss=1.5059 acc=0.4720
[small_capacity] Epoch 13 | train_loss=1.3926 acc=0.5005 |
val_loss=1.4639 acc=0.4894
[small_capacity] Epoch 14 | train_loss=1.3805 acc=0.5041 |
val_loss=1.4833 acc=0.4820
[small_capacity] Epoch 15 | train_loss=1.3790 acc=0.5034 |
val_loss=1.4685 acc=0.4800
[small_capacity] Epoch 16 | train_loss=1.3621 acc=0.5091 |
```

```
val_loss=1.4670 acc=0.4934
[small_capacity] Epoch 17 | train_loss=1.3589 acc=0.5106 |
val_loss=1.4581 acc=0.4852
[small_capacity] Epoch 18 | train_loss=1.3546 acc=0.5126 |
val_loss=1.4491 acc=0.4846
[small_capacity] Epoch 19 | train_loss=1.3441 acc=0.5159 |
val_loss=1.4673 acc=0.4810
[small_capacity] Epoch 20 | train_loss=1.3366 acc=0.5212 |
val_loss=1.4445 acc=0.4882
[small_capacity] Epoch 21 | train_loss=1.3260 acc=0.5217 |
val_loss=1.4590 acc=0.4910
[small_capacity] Epoch 22 | train_loss=1.3251 acc=0.5239 |
val_loss=1.4848 acc=0.4844
[small_capacity] Epoch 23 | train_loss=1.3181 acc=0.5282 |
val_loss=1.4548 acc=0.4888
[small_capacity] Epoch 24 | train_loss=1.3169 acc=0.5262 |
val_loss=1.4434 acc=0.4964
[small_capacity] Epoch 25 | train_loss=1.3091 acc=0.5288 |
val_loss=1.4569 acc=0.4862
[small_capacity] Epoch 26 | train_loss=1.3052 acc=0.5311 |
val_loss=1.4379 acc=0.4934
[small_capacity] Epoch 27 | train_loss=1.3041 acc=0.5309 |
val_loss=1.4448 acc=0.4924
[small_capacity] Epoch 28 | train_loss=1.2977 acc=0.5350 |
val_loss=1.4523 acc=0.4962
[small_capacity] Epoch 29 | train_loss=1.2922 acc=0.5362 |
val_loss=1.4269 acc=0.5036
[small_capacity] Epoch 30 | train_loss=1.2861 acc=0.5379 |
val_loss=1.4292 acc=0.5018

Ringkasan Model Kecil: {'exp_name': 'small_capacity', 'best_epoch':
29, 'best_val_loss': 1.4269202249526978, 'best_val_acc': 0.5036,
'training_time': 498.3623094558716, 'epochs_run': 30, 'num_params':
820874}

set_seed(SEED)
model_large = MLP(hidden_layers=(1024,512,256), dropout=0.0,
use_bn=False).to(device)
opt_large = optim.Adam(model_large.parameters(), lr=0.001)
model_large, hist_large, res_large = run_training(
    model_large, train_loader, val_loader, opt_large,
    scheduler=None, epochs=30, patience=5, exp_name="large_capacity"
)
res_large["num_params"] = count_params(model_large)
print("\nRingkasan Model Besar:", res_large)

[large_capacity] Epoch 01 | train_loss=1.8268 acc=0.3377 |
val_loss=1.7215 acc=0.3812
[large_capacity] Epoch 02 | train_loss=1.6494 acc=0.4015 |
val_loss=1.6844 acc=0.4018
```

```
[large_capacity] Epoch 03 | train_loss=1.5728 acc=0.4324 |
val_loss=1.6616 acc=0.4052
[large_capacity] Epoch 04 | train_loss=1.5232 acc=0.4490 |
val_loss=1.6341 acc=0.4330
[large_capacity] Epoch 05 | train_loss=1.4823 acc=0.4640 |
val_loss=1.5600 acc=0.4476
[large_capacity] Epoch 06 | train_loss=1.4516 acc=0.4762 |
val_loss=1.5492 acc=0.4486
[large_capacity] Epoch 07 | train_loss=1.4319 acc=0.4825 |
val_loss=1.5080 acc=0.4588
[large_capacity] Epoch 08 | train_loss=1.4046 acc=0.4946 |
val_loss=1.5937 acc=0.4548
[large_capacity] Epoch 09 | train_loss=1.3876 acc=0.5003 |
val_loss=1.5737 acc=0.4456
[large_capacity] Epoch 10 | train_loss=1.3729 acc=0.5076 |
val_loss=1.5246 acc=0.4678
[large_capacity] Epoch 11 | train_loss=1.3557 acc=0.5112 |
val_loss=1.4967 acc=0.4808
[large_capacity] Epoch 12 | train_loss=1.3435 acc=0.5176 |
val_loss=1.4959 acc=0.4770
[large_capacity] Epoch 13 | train_loss=1.3277 acc=0.5233 |
val_loss=1.5106 acc=0.4662
[large_capacity] Epoch 14 | train_loss=1.3141 acc=0.5296 |
val_loss=1.4835 acc=0.4854
[large_capacity] Epoch 15 | train_loss=1.3058 acc=0.5311 |
val_loss=1.5025 acc=0.4684
[large_capacity] Epoch 16 | train_loss=1.3007 acc=0.5320 |
val_loss=1.5098 acc=0.4830
[large_capacity] Epoch 17 | train_loss=1.2875 acc=0.5378 |
val_loss=1.4514 acc=0.4832
[large_capacity] Epoch 18 | train_loss=1.2729 acc=0.5433 |
val_loss=1.4801 acc=0.4728
[large_capacity] Epoch 19 | train_loss=1.2669 acc=0.5422 |
val_loss=1.4943 acc=0.4776
[large_capacity] Epoch 20 | train_loss=1.2631 acc=0.5453 |
val_loss=1.5518 acc=0.4606
[large_capacity] Epoch 21 | train_loss=1.2550 acc=0.5494 |
val_loss=1.4842 acc=0.4770
[large_capacity] Epoch 22 | train_loss=1.2499 acc=0.5492 |
val_loss=1.5295 acc=0.4718
Early stopping dipicu pada epoch 22 (patience=5)

Ringkasan Model Besar: {'exp_name': 'large_capacity', 'best_epoch':
17, 'best_val_loss': 1.4514257610321044, 'best_val_acc': 0.4854,
'training_time': 367.77381587028503, 'epochs_run': 22, 'num_params':
3805450}
```

19. Perbandingan Hasil Seluruh Eksperimen

Menggabungkan hasil semua eksperimen ke dalam satu `DataFrame` agar mudah dibandingkan. Kolom `test_accuracy` diisi belakangan setelah evaluasi test set (bagian 21) dijalankan untuk model terbaik saja — jika ingin test_accuracy tiap eksperimen, panggil `evaluate_on_test` (didefinisikan di bagian 21) untuk masing-masing model.

```
res_baseline["num_params"] = count_params(model_baseline)
res_sgd["num_params"] = count_params(model_sgd)
res_step["num_params"] = count_params(model_step)
res_dropout["num_params"] = count_params(model_dropout)
res_bn["num_params"] = count_params(model_bn)
res_dropbn["num_params"] = count_params(model_dropbn)

config_info = {
    "baseline":          dict(optimizer="Adam",  scheduler="-",
dropout=0.0, batch_norm=False, hidden_layers="512,256"),
    "sgd_momentum":      dict(optimizer="SGD+Momentum",
scheduler="-", dropout=0.0, batch_norm=False,
hidden_layers="512,256"),
    "steplr":            dict(optimizer="Adam",  scheduler="StepLR",
dropout=0.0, batch_norm=False, hidden_layers="512,256"),
    "dropout":           dict(optimizer="Adam",  scheduler="-",
dropout=0.3, batch_norm=False, hidden_layers="512,256"),
    "batchnorm":         dict(optimizer="Adam",  scheduler="-",
dropout=0.0, batch_norm=True,  hidden_layers="512,256"),
    "dropout_batchnorm": dict(optimizer="Adam",  scheduler="-",
dropout=0.3, batch_norm=True,  hidden_layers="512,256"),
    "small_capacity":    dict(optimizer="Adam",  scheduler="-",
dropout=0.0, batch_norm=False, hidden_layers="256,128"),
    "large_capacity":    dict(optimizer="Adam",  scheduler="-",
dropout=0.0, batch_norm=False, hidden_layers="1024,512,256"),
}

all_results = [res_baseline, res_sgd, res_step, res_dropout, res_bn,
res_dropbn, res_small, res_large]

rows = []
for r in all_results:
    cfg = config_info[r["exp_name"]]
    rows.append({
        "experiment": r["exp_name"],
        "optimizer": cfg["optimizer"],
        "scheduler": cfg["scheduler"],
        "dropout": cfg["dropout"],
        "batch_norm": cfg["batch_norm"],
        "hidden_layers": cfg["hidden_layers"],
        "epochs_run": r["epochs_run"],
        "best_epoch": r["best_epoch"],
```

```
        "best_val_loss": r["best_val_loss"],
        "best_val_accuracy": r["best_val_acc"],
        "training_time_sec": r["training_time"],
        "num_params": r.get("num_params"),
    })

results_df = pd.DataFrame(rows)
results_df
```


```
{"summary":"{\n  \"name\": \"results_df\",\n  \"rows\": 8,\n
\"fields\": [\n    {\n      \"column\": \"experiment\",\n
\"properties\": {\n        \"dtype\": \"string\",\n
\"num_unique_values\": 8,\n        \"samples\": [\n
\"sgd_momentum\",\n          \"dropout_batchnorm\",\n
\"baseline\"\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"optimizer\",\n      \"properties\": {\n        \"dtype\":
\"category\",\n        \"num_unique_values\": 2,\n        \"samples\":
[\n          \"SGD+Momentum\",\n          \"Adam\"\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"scheduler\",\n
\"properties\": {\n        \"dtype\": \"category\",\n
\"num_unique_values\": 2,\n        \"samples\": [\n
\"StepLR\",\n          \"-\"\n        ],\n        \"semantic_type\":
\"\",\n        \"description\": \"\"\n      }\n    },\n    {\n
\"column\": \"dropout\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 0.13887301496588272,\n        \"min\":
0.0,\n        \"max\": 0.3,\n        \"num_unique_values\": 2,\n
\"samples\": [\n          0.3,\n          0.0\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"batch_norm\",\n
\"properties\": {\n        \"dtype\": \"boolean\",\n
\"num_unique_values\": 2,\n        \"samples\": [\n          true,\n
false\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"hidden_layers\",\n      \"properties\": {\n        \"dtype\":
\"category\",\n        \"num_unique_values\": 3,\n        \"samples\":
[\n          \"512,256\",\n          \"256,128\"\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"epochs_run\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
7,\n        \"min\": 11,\n        \"max\": 30,\n
\"num_unique_values\": 5,\n        \"samples\": [\n          19,\n
22\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"best_epoch\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 8,\n        \"min\": 6,\n
\"max\": 30,\n        \"num_unique_values\": 7,\n        \"samples\":
[\n          6,\n          14\n        ],\n        \"semantic_type\":
\"\",\n        \"description\": \"\"\n      }\n    },\n    {\n
```

\"column\": \"best_val_loss\",\n   \"properties\": {\n
\"dtype\": \"number\",\n   \"std\": 0.11764639555887517,\n
\"min\": 1.2011934938430786,\n   \"max\": 1.5624053495407104,\n
\"num_unique_values\": 8,\n   \"samples\": [\n
1.380197254562378,\n   1.2988003002166748\n   ],\n
\"semantic_type\": \"\",\n   \"description\": \"\"\n   }\
n   },\n   {\n   \"column\": \"best_val_accuracy\",\n
\"properties\": {\n   \"dtype\": \"number\",\n   \"std\":
0.04181707614292104,\n   \"min\": 0.4496,\n   \"max\":
0.579,\n   \"num_unique_values\": 8,\n   \"samples\": [\n
0.5172,\n   0.5376\n   ],\n   \"semantic_type\":
\"\",\n   \"description\": \"\"\n   }\n   },\n   {\n
\"column\": \"training_time_sec\",\n   \"properties\": {\n
\"dtype\": \"number\",\n   \"std\": 120.38751121821174,\n
\"min\": 179.3498923778534,\n   \"max\": 512.6446645259857,\n
\"num_unique_values\": 8,\n   \"samples\": [\n
309.85204100608826,\n   506.6082389354706\n   ],\n
\"semantic_type\": \"\",\n   \"description\": \"\"\n   }\
n   },\n   {\n   \"column\": \"num_params\",\n
\"properties\": {\n   \"dtype\": \"number\",\n   \"std\":
845455,\n   \"min\": 820874,\n   \"max\": 3805450,\n
\"num_unique_values\": 4,\n   \"samples\": [\n   1708810,\
n   3805450\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   }\n]\
n}","type":"dataframe","variable_name":"results_df"}

```
fig, axes = plt.subplots(2, 2, figsize=(14,10))

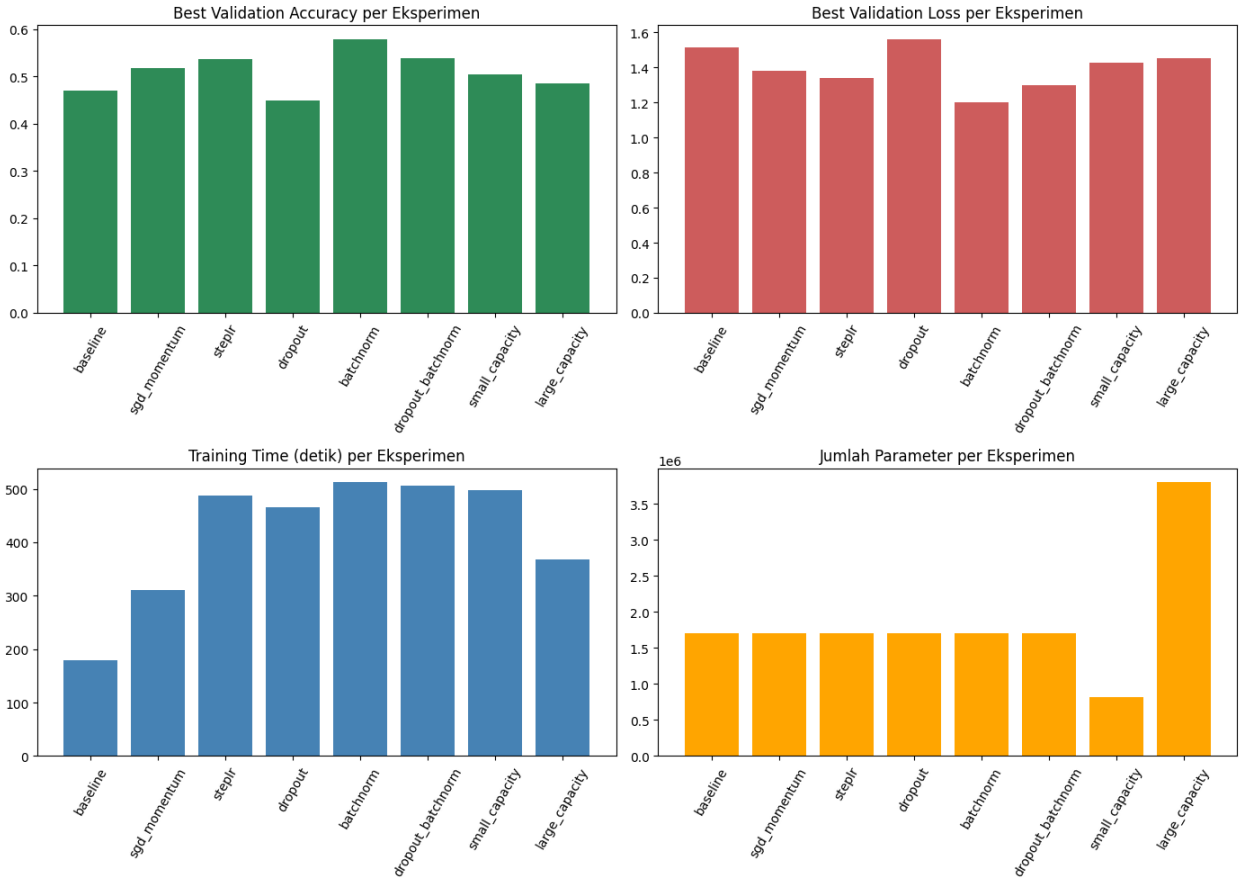
axes[0,0].bar(results_df["experiment"],
results_df["best_val_accuracy"], color="seagreen")
axes[0,0].set_title("Best Validation Accuracy per Eksperimen")
axes[0,0].tick_params(axis='x', rotation=60)

axes[0,1].bar(results_df["experiment"], results_df["best_val_loss"],
color="indianred")
axes[0,1].set_title("Best Validation Loss per Eksperimen")
axes[0,1].tick_params(axis='x', rotation=60)

axes[1,0].bar(results_df["experiment"],
results_df["training_time_sec"], color="steelblue")
axes[1,0].set_title("Training Time (detik) per Eksperimen")
axes[1,0].tick_params(axis='x', rotation=60)

axes[1,1].bar(results_df["experiment"], results_df["num_params"],
color="orange")
axes[1,1].set_title("Jumlah Parameter per Eksperimen")
axes[1,1].tick_params(axis='x', rotation=60)

plt.tight_layout()
plt.show()
```



20. Pemilihan Model Terbaik

Model terbaik TIDAK dipilih hanya berdasarkan validation accuracy tertinggi, melainkan mempertimbangkan: validation performance, indikasi overfitting (gap train-val), training time, dan kompleksitas model (jumlah parameter).

Langkah: lihat tabel `results_df` di atas setelah dijalankan, lalu tentukan `best_exp_name` secara manual berdasarkan bukti (isi variabel di sel berikut sesuai hasil nyata Anda).

```
# TODO: Setelah melihat results_df, isi nama eksperimen dengan performa terbaik berdasarkan
# bukti (bukan hanya validation accuracy tertinggi, tapi juga overfitting & training time).
best_exp_name = "baseline"  # ganti sesuai hasil eksperimen Anda, misal "dropout_batchnorm"

model_map = {
    "baseline": model_baseline,
    "sgd_momentum": model_sgd,
    "steplr": model_step,
    "dropout": model_dropout,
    "batchnorm": model_bn,
    "dropout_batchnorm": model_dropbn,
```

```
    "small_capacity": model_small,
    "large_capacity": model_large,
}
history_map = {
    "baseline": hist_baseline,
    "sgd_momentum": hist_sgd,
    "steplr": hist_step,
    "dropout": hist_dropout,
    "batchnorm": hist_bn,
    "dropout_batchnorm": hist_dropbn,
    "small_capacity": hist_small,
    "large_capacity": hist_large,
}

best_model = model_map[best_exp_name]
best_history = history_map[best_exp_name]
print("Model terbaik yang dipilih:", best_exp_name)
```

```
Model terbaik yang dipilih: baseline
```

21. Evaluasi pada Test Set

Evaluasi HANYA dilakukan sekali, pada model terbaik yang sudah dipilih, memakai `test_loader` yang belum pernah dilihat model selama training/pemilihan model.

```
@torch.no_grad()
def evaluate_on_test(model, loader):
    model.eval()
    criterion = nn.CrossEntropyLoss()
    total_loss, correct, total = 0.0, 0, 0
    all_preds, all_labels = [], []
    for x, y in loader:
        x, y = x.to(device), y.to(device)
        out = model(x)
        loss = criterion(out, y)
        total_loss += loss.item() * x.size(0)
        preds = out.argmax(1)
        correct += (preds == y).sum().item()
        total += x.size(0)
        all_preds.append(preds.cpu().numpy())
        all_labels.append(y.cpu().numpy())
    return total_loss/total, correct/total, np.concatenate(all_preds),
np.concatenate(all_labels)

test_loss, test_acc, y_pred, y_true = evaluate_on_test(best_model,
test_loader)
print(f"Model terbaik: {best_exp_name}")
print(f"Test Loss    : {test_loss:.4f}")
print(f"Test Accuracy: {test_acc:.4f}")
```

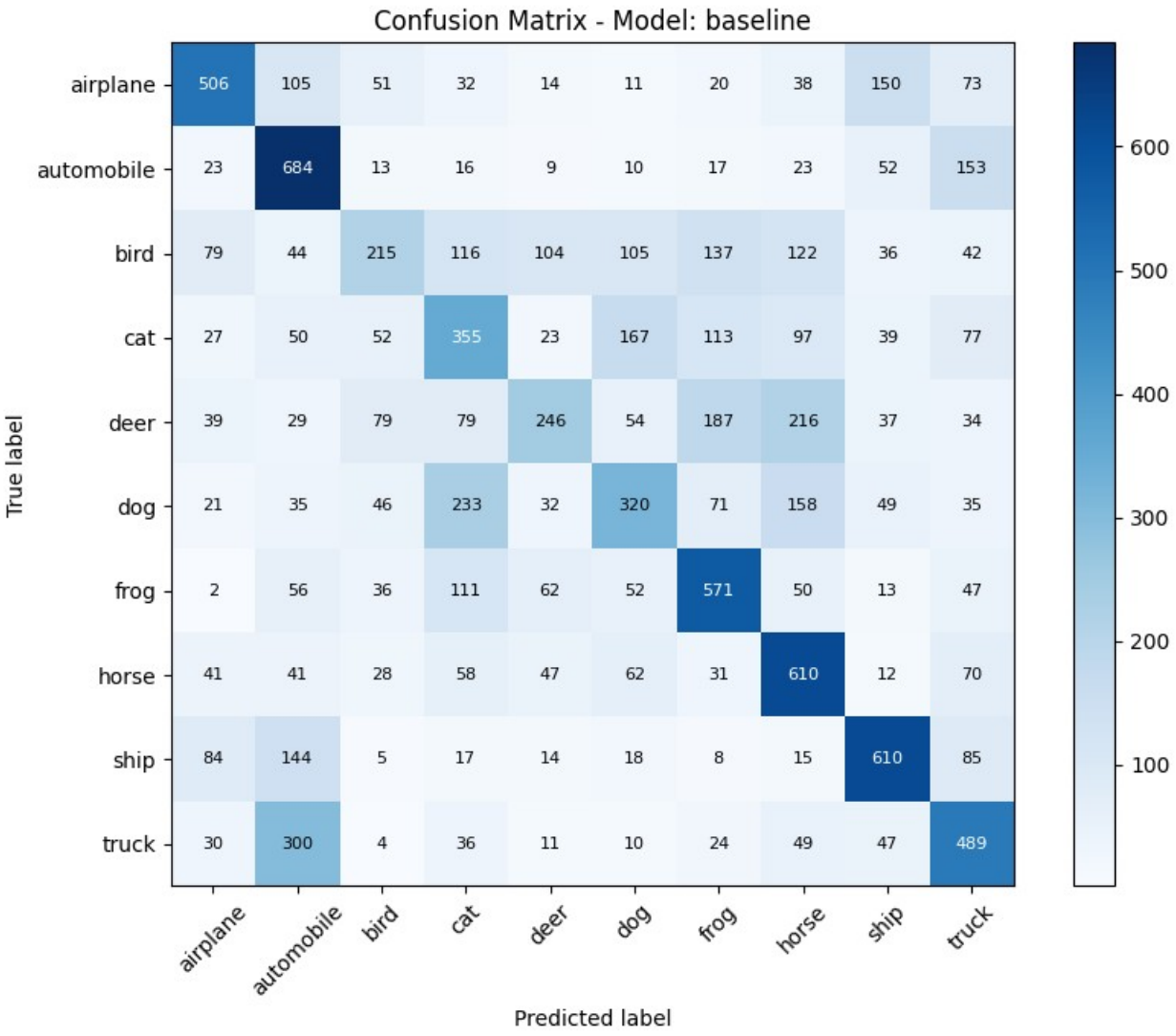
```
Model terbaik: baseline
Test Loss    : 1.5154
Test Accuracy: 0.4606
```

22. Confusion Matrix

Menampilkan confusion matrix dengan nama kelas, untuk melihat kelas mana yang paling sering tertukar oleh model.

```
cm = confusion_matrix(y_true, y_pred)

plt.figure(figsize=(9,7))
plt.imshow(cm, cmap="Blues")
plt.title(f"Confusion Matrix - Model: {best_exp_name}")
plt.colorbar()
plt.xticks(range(10), CLASSES, rotation=45)
plt.yticks(range(10), CLASSES)
plt.xlabel("Predicted label")
plt.ylabel("True label")
for i in range(10):
    for j in range(10):
        plt.text(j, i, cm[i,j], ha="center", va="center",
                 color="white" if cm[i,j] > cm.max()/2 else "black",
fontsize=8)
plt.tight_layout()
plt.show()
```



23. Classification Report

Menampilkan precision, recall, dan f1-score per kelas.

```
print(classification_report(y_true, y_pred, target_names=CLASSES))
```

```
              precision    recall  f1-score   support

    airplane       0.59      0.51      0.55      1000
  automobile       0.46      0.68      0.55      1000
        bird       0.41      0.21      0.28      1000
         cat       0.34      0.35      0.35      1000
        deer       0.44      0.25      0.31      1000
         dog       0.40      0.32      0.35      1000
        frog       0.48      0.57      0.52      1000
       horse       0.44      0.61      0.51      1000
```

```
        ship       0.58      0.61      0.60      1000
       truck       0.44      0.49      0.46      1000

    accuracy                           0.46     10000
   macro avg       0.46      0.46      0.45     10000
weighted avg       0.46      0.46      0.45     10000
```

24. Grafik Train vs Validation (Model Terbaik)

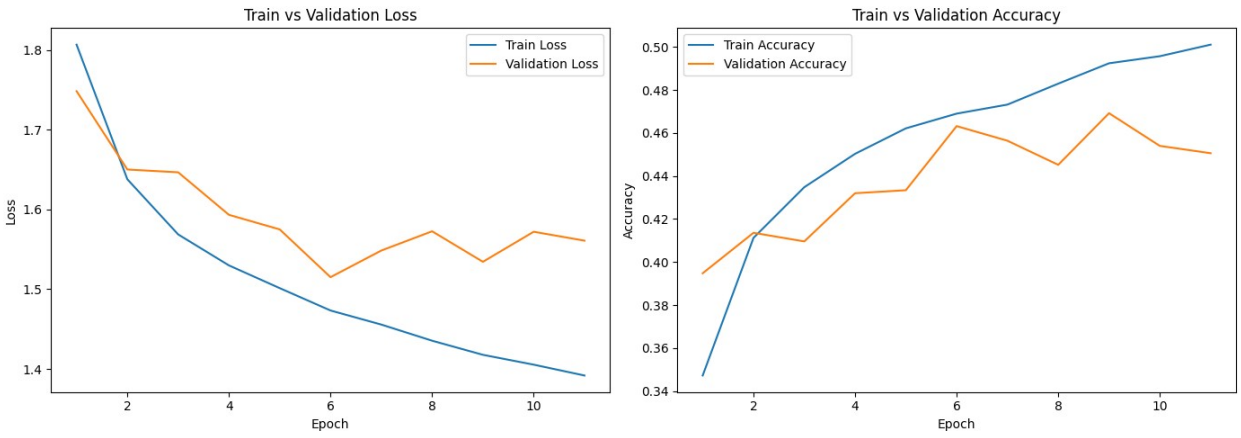
Digunakan untuk menilai apakah model overfitting, underfitting, atau sudah convergence.

```
epochs_range = range(1, len(best_history["train_loss"])+1)

fig, axes = plt.subplots(1, 2, figsize=(14,5))
axes[0].plot(epochs_range, best_history["train_loss"], label="Train
Loss")
axes[0].plot(epochs_range, best_history["val_loss"], label="Validation
Loss")
axes[0].set_title("Train vs Validation Loss")
axes[0].set_xlabel("Epoch"); axes[0].set_ylabel("Loss");
axes[0].legend()

axes[1].plot(epochs_range, best_history["train_acc"], label="Train
Accuracy")
axes[1].plot(epochs_range, best_history["val_acc"], label="Validation
Accuracy")
axes[1].set_title("Train vs Validation Accuracy")
axes[1].set_xlabel("Epoch"); axes[1].set_ylabel("Accuracy");
axes[1].legend()

plt.tight_layout()
plt.show()
```



25. Analisis Hasil

> **Isi bagian ini SETELAH menjalankan seluruh notebook**, berdasarkan angka & grafik yang benar-benar muncul. Jangan menuliskan interpretasi yang tidak didukung grafik/tabel.

Beberapa pertanyaan pemandu (jawab berdasarkan hasil nyata Anda):

- **Optimizer:** Adam vs SGD+Momentum, mana yang mencapai validation loss/accuracy lebih baik dan lebih cepat convergence? Mengapa (kaitkan dengan cara kerja masing-masing optimizer)?
- **Scheduler:** Apakah StepLR membantu konvergensi dibanding tanpa scheduler? Lihat apakah val_loss lebih stabil di epoch-epoch akhir.
- **Regularisasi:** Bandingkan gap antara train_acc dan val_acc pada baseline vs baseline+dropout vs baseline+batchnorm vs keduanya. Apakah gap mengecil (indikasi overfitting berkurang)?
- **Kapasitas model:** Apakah model besar benar-benar lebih baik, atau hanya menambah waktu training tanpa peningkatan akurasi validasi yang signifikan (overfitting)?
- **Confusion Matrix:** Kelas mana yang paling sering tertukar (misalnya cat vs dog, atau automobile vs truck)? Mengapa masuk akal secara visual (kemiripan bentuk/warna)?

(Tulis analisis Anda di sini setelah melihat output asli.)

26. Kesimpulan

> **Isi bagian ini SETELAH seluruh eksperimen selesai dijalankan**, dengan format berbasis bukti, misalnya:

"Model **`<nama_eksperimen_terbaik>`** dipilih sebagai model terbaik karena memperoleh validation accuracy tertinggi sebesar **`<isi>`**, test accuracy sebesar **`<isi>`**, dengan training time **`<isi>`** detik, dan gap train-validation accuracy sebesar **`<isi>`** (mengindikasikan `<overfitting ringan/tidak overfitting/dst>`). Dibandingkan model dengan kapasitas lebih besar, model ini dipilih karena `<alasan efisiensi/generalisasi>`."

Ringkasan capaian pembelajaran:

- Baseline ANN (3072→512→256→10) berhasil dilatih menggunakan CrossEntropyLoss dan dievaluasi dengan metrik loss & akurasi.
- Early stopping dan model checkpointing berhasil diimplementasikan from scratch berbasis validation loss.
- Data augmentasi (RandomCrop, RandomHorizontalFlip) diterapkan hanya pada data training.
- Eksperimen sistematis (optimizer, scheduler, regularisasi, kapasitas model) dijalankan secara controlled (satu variabel diubah setiap kali).
- Model terbaik dipilih berdasarkan bukti (validation performance, overfitting, training time, kompleksitas), bukan hanya test accuracy.

(Lengkapi dengan angka-angka nyata dari hasil eksekusi Anda.)